

1. Understand String Immutability (Interview Revision)

What is String Immutability?

A **String** object cannot be changed after it is created.

```
String s = "Java";  
  
s.concat(" Programming");  
  
System.out.println(s);
```

Output

```
Java
```

Why?

Because `concat()` creates a **new String object**. The original "Java" remains unchanged.

Correct way:

```
s = s.concat(" Programming");
```

Why Did Java Make Strings Immutable?

1. Security ★ (Most Asked)

Strings are used in:

- Database URLs
- File paths
- Usernames/passwords
- Network connections

If Strings were mutable, malicious code could change them after validation.

2. String Pool Optimization

Many variables can share the same String object.

```
String a = "Java";  
String b = "Java";
```

Both reference the same pooled object.

Immutability makes this safe.

3. Thread Safety

Multiple threads can safely share the same String object because it cannot change.

No synchronization is required.

4. Hashing Performance

String is heavily used as a key in HashMap.

Since it never changes, its hash code can be cached.

This improves performance.

Interviewer's Reasoning

Instead of asking:

Why is String immutable?

They're checking whether you understand Java's design.

A strong answer is:

"String is immutable for security, thread safety, String Pool optimization, and efficient hashing. Since the object cannot change, it can be safely shared across different parts

of the application."

Common Misconception



```
s.concat(" Java");
```

changes s.



It creates a new object.

Comparison

Mutable	Immutable
StringBuilder	String
StringBuffer	String
ArrayList	String

Tricky Interview Questions

Q1. Does concat() modify the original String?

No.

Q2. Why is StringBuilder mutable but String isn't?

StringBuilder is designed for **frequent modifications**, while String is optimized for **safety and sharing**.

Q3. Can an immutable object ever change?

No.

Any "modification" creates a **new object**.

Q4. If String is immutable, why is this valid?

```
String s = "Java";  
  
s = "Python";
```

Because the **reference changes**, not the original String object.

Interview Tip

A common interview follow-up is:

If String is immutable, why does `s = "Python"` work?

Expected answer:

"The original String object isn't modified. The variable `s` is simply updated to reference a different String object."

2. Common String Methods (Quick Revision)

`length()`

Returns the number of characters.

```
String s = "Java";  
  
System.out.println(s.length());
```

Output

```
4
```

Interview Trap

length vs length()

```
arr.length
```

Property

```
str.length()
```

Method

charAt()

Returns a character at a given index.

```
String s = "Java";  
  
System.out.println(s.charAt(2));
```

Output

```
v
```

Trap

```
s.charAt(s.length());
```

Throws

```
StringIndexOutOfBoundsException
```

Correct

```
s.charAt(s.length()-1)
```

substring()

Extracts part of a String.

```
String s = "Programming";  
System.out.println(s.substring(3));
```

Output

```
gramming
```

```
System.out.println(s.substring(3,7));
```

Output

```
gram
```

Interview Reasoning

Second parameter is **exclusive**, not inclusive.

contains()

Checks if a substring exists.

```
String s="Java Programming";  
System.out.println(s.contains("Java"));
```

Output

```
true
```

Interview Trap

Case-sensitive.

```
"Java".contains("java")
```

returns

```
false
```

trim()

Removes leading and trailing whitespace.

```
String s="  Java  ";  
System.out.println(s.trim());
```

Output

Java

Interview Trap

`trim()` **does not remove spaces in the middle.**

"Java Programming"

remains unchanged internally.

Interview Comparison

Method	Purpose
<code>length()</code>	Count characters
<code>charAt()</code>	Access one character
<code>substring()</code>	Extract part of String
<code>contains()</code>	Search text
<code>trim()</code>	Remove leading/trailing spaces

Tricky Output Questions

Question 1

```
String s="Java";  
  
System.out.println(s.charAt(s.length()-1));
```

Output

a

Question 2

```
String s="Programming";  
System.out.println(s.substring(4,4));
```

Output

""

(Empty String)

Question 3

```
String s=" Java ";  
System.out.println(s.length());  
System.out.println(s.trim().length());
```

Output

6
4

Question 4

```
String s="Java";  
System.out.println(s.contains("va"));
```

Output

```
true
```

Question 5

```
String s="Java";  
  
System.out.println(s.substring(1));
```

Output

```
ava
```

Top Interview Questions

String Immutability

1. Why is String immutable?
 2. Does concat() modify the original String?
 3. Why does Java use immutable Strings?
 4. Difference between String and StringBuilder?
 5. Why is immutability useful in HashMap?
-

String Methods

1. Difference between length and length()?
 2. Is substring()'s ending index inclusive?
 3. Does trim() remove all spaces?
 4. Is contains() case-sensitive?
 5. What exception does charAt() throw for an invalid index?
-

★ Interview Insight (Very Common)

A senior interviewer rarely asks, "**What does substring() do?**"

Instead, they ask questions like:

- **Why doesn't trim() remove spaces in the middle?**
- **Why is the second index in substring(begin, end) exclusive?**
- **Why is charAt(length()) invalid but charAt(length() - 1) valid?**
- **Why does concat() return a new object instead of modifying the original?**

These questions test whether you understand **the design and behavior** of the String API, not just its syntax. Explaining the reasoning behind these methods will make your answers stand out in Java internship interviews.